the Authorization Code Grant with PKCE (Proof Key for Code Exchange).

- **Resource Owner Password Credentials Grant.** In this flow, the user directly provides their username and password to the client application, which exchanges them for a token. This essentially bypasses OAuth's protections and is safe only for first-party clients that the user already trusts with their credentials. For third-party apps, it reintroduces the password anti-pattern that OAuth was designed to eliminate.

These flows still exist in legacy systems, but ACME uses the Authorization Code and Client Credentials grants for new integrations. They support both interactive user-driven access (like Kate and BI Explorer) and long-running service-to-service integrations (such as CustCo's Task App).

11.2.4 Using OAuth tokens

No matter how an access token is obtained—whether through the Authorization Code Grant, the Client Credentials Grant, or one of the secondary flows—the process of using that token remains the same. Tokens are how a client proves it has been granted access. Each API request includes the token, and every request is verified before it reaches the underlying service.

At ACME, OAuth provides only the broad perimeter: a token says, for example, "this client can read document metadata" or "this integration can write tasks." Cedar policies then supply the fine-grained access control, ensuring the token is applied only to the appropriate resources under the correct conditions. That means Kate's BI Explorer token allows her to read metadata, but only for workspaces she belongs to. CustCo's Task App can use the token to write tasks only in the specific CustCo workspaces approved during installation. Simultaneously, Cedar can enforce additional rules, such as excluding sensitive document types or requiring user attribution for writes. Figure 11.8 illustrates how an access token is used.